

**ESCUELA POLITÉCNICA
SUPERIOR DE ZAMORA
UNIVERSIDAD DE SALAMANCA**

PRÁCTICAS DE SISTEMAS OPERATIVOS II

PRÁCTICA WINDOWS DE GRUPO

Aparcando II

Documentación de Sincronización

Grupo: G08

Curso: 2025-2026

Integrantes:

- Brais Bértolo Senra
- Juan Riego Vila

Fecha: Mayo de 2026

Contents

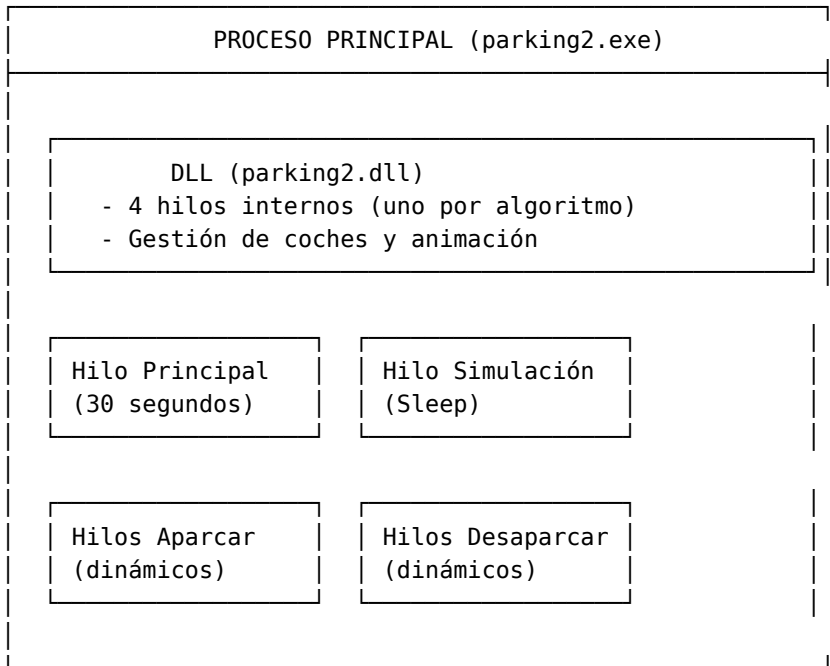
1. Introducción	4
2. Diagrama General del Sistema	4
3. Recursos y Objetos de Sincronización	4
3.1. Objetos de Sincronización por Algoritmo	4
3.2. Estructuras de Datos Compartidas	5
4. Hilos del Sistema	5
4.1. Hilo Principal (main)	5
4.2. Hilo de Simulación Principal (hilo_principal_simulacion)	6
4.3. Hilos de Aparcamiento (hilo_aparcar)	6
4.4. Hilos de Desaparcamiento (hilo_desaparcar)	6
5. Pseudocódigo de Operaciones Críticas	7
5.1. Secuencia de Aparcamiento	7
5.2. Callback: aparcar_commit	7
5.3. Callback: permiso_avance	8
5.4. Callback: permiso_avance_commit	8
6. Algoritmos de Búsqueda	9
6.1. Primer Ajuste	9
6.2. Siguiente Ajuste	10
6.3. Mejor Ajuste	10
6.4. Peor Ajuste	11
7. Tabla Resumen de Objetos de Sincronización	12
8. Consideraciones de Concurrencia	12
8.1. Puntos Críticos	12
8.2. Garantías de Sincronización	12
9. Optimizaciones Realizadas	12
9.1. 1. Salto de Coches en Arrays	12
9.2. 2. Retroceso en Siguiente Ajuste	13
9.3. 3. Búsqueda Circular en Siguiente Ajuste	13
9.4. 4. Cierre Inmediato de Manejadores de Hilos	13
10. Ejemplo de Ejecución	13
10.1. Escenario: Tres Coches Llegando	13

1. Introducción

Este documento describe la arquitectura de sincronización implementada en la práctica Windows “Aparcando II”. Se detalla la organización de recursos, objetos de sincronización, estructura de hilos y el pseudocódigo de las operaciones más críticas.

2. Diagrama General del Sistema

El sistema está basado en un **único proceso** con **múltiples hilos**, a diferencia de la versión Linux que utilizaba múltiples procesos.



3. Recursos y Objetos de Sincronización

3.1. Objetos de Sincronización por Algoritmo

Para cada uno de los 4 algoritmos se crean los siguientes objetos:

Para cada algoritmo $i \in \{0, 1, 2, 3\}$

(PRIMER_AJUSTE, SIGUIENTE_AJUSTE, MEJOR_AJUSTE, PEOR_AJUSTE):

```
HANDLE hMutex[i]      // MUTEX
HANDLE hOrden[i]      // SEMÁFORO
HANDLE hAvance[i]     // SEMÁFORO
```

3.1.1. hMutex[i] - Mutex (Exclusión Mutua)

- **Tipo:** HANDLE (Mutex)
- **Valor Inicial:** Libre (sin propietario)
- **Función:** Proteger acceso exclusivo a las estructuras de datos compartidas (arrays **acera** y **carril**)
- **Uso:**
 - WaitForSingleObject(hMutex[i], INFINITE) - Adquirir
 - ReleaseMutex(hMutex[i]) - Liberar

3.1.2. hOrden[i] - Semáforo de Orden

- **Tipo:** HANDLE (Semáforo)
- **Valor Inicial:** 1 (permiso para que aparque el coche número 1)

- **Rango:** 0 a 1 (solo puede pasar un coche a la vez)
- **Función:** Garantizar que los coches aparquen en orden numérico consecutivo (1, 2, 3, ...)
- **Uso:**
 - En `hilo_aparcar()`: Esperar con `WaitForSingleObject()` antes de aparcar
 - En `aparcar_commit()`: Incrementar con `ReleaseSemaphore(..., 1, NULL)` para permitir al siguiente

3.1.3. `hAvance[i]` - Semáforo de Avance

- **Tipo:** `HANDLE` (Semáforo)
- **Valor Inicial:** 0 (bloqueado)
- **Rango:** 0 a 999 (permite múltiples coches esperando)
- **Función:** Coordinar el avance seguro de coches cuando hay contienda por posiciones en el carril
- **Uso:**
 - En `permiso_avance()`: Si no hay espacio, esperar con `WaitForSingleObject()`
 - En `permiso_avance_commit()`: Despertar bloqueados con `ReleaseSemaphore(..., 1, NULL)`

3.2. Estructuras de Datos Compartidas

```
// Arrays que almacenan el estado de la acera para cada algoritmo
int acera[NUM_ALGORITMOS][TAM_PARKING]; // 4 × 80 = 320 posiciones
int carril[NUM_ALGORITMOS][TAM_PARKING]; // 4 × 80 = 320 posiciones

// Control de orden de aparcamiento por algoritmo
int proxAparcarNum[NUM_ALGORITMOS]; // Próximo número de coche que puede aparcar

// Control de posición inicial para siguiente_ajuste
int proxAparcar[NUM_ALGORITMOS]; // Próxima posición a buscar
```

3.2.1. `acera[alg][pos]`

- **Rango de pos:** 0 a 79
- **Valor cuando está libre:** 0
- **Valor cuando está ocupada:** Longitud del coche que la ocupa
- **Propósito:** Almacenar qué posiciones de la acera están ocupadas y por coches de qué longitud
- **Protección:** Acceso desde funciones de llegada, sin mutex (lectura rápida)

3.2.2. `carril[alg][pos]`

- **Rango de pos:** 0 a 79
- **Valor cuando está libre:** 0
- **Valor cuando está ocupada:** 1
- **Propósito:** Almacenar qué posiciones del carril están ocupadas durante los movimientos
- **Protección:** Acceso protegido por `hMutex[alg]`

3.2.3. `proxAparcarNum[alg]`

- **Tipo:** `int`
- **Valor Inicial:** 1
- **Rango:** Número de coche esperado que debe aparcar siguiente
- **Propósito:** Garantizar orden numérico en aparcamientos
- **Protección:** Acceso protegido por `hOrden[alg]`

4. Hilos del Sistema

4.1. Hilo Principal (`main`)

```
main()
└─ SetConsoleCtrlHandler() // Registrar manejador Ctrl+C
```

```

└─ cargar_dll() // Cargar parking2.dll
└─ inicializar_sincronizacion() // Crear mutex y semáforos
└─ PARKING2_inicio() // Iniciar simulación DLL
└─ CreateThread(hilo_principal_simulacion) // Crear hilo de 30s
└─ WaitForSingleObject() // Esperar finalización
└─ liberar_sincronizacion() // Liberar objetos
└─ eliminar_dll() // Descargar DLL
└─ return 0

```

4.2. Hilo de Simulación Principal (hilo_principal_simulacion)

```

hilo_principal_simulacion(param)
└─ Sleep(30000) // Dormir 30 segundos
└─ terminar = 1
└─ PARKING2_fin() // Terminar simulación
└─ return 0

```

Duración: 30 segundos máximo **Propósito:** Controlar el tiempo total de la simulación

4.3. Hilos de Aparcamiento (hilo_aparcar)

Se crean dinámicamente cuando un coche necesita aparcar.

```

hilo_aparcar(DATOS_HILO *datos)
└─ HCoche hc ← datos->hc
└─ int alg ← datos->algoritmo
└─
└─ WaitForSingleObject(hOrden[alg], INFINITE)
    └─ Esperar a que sea el turno de este coche
└─
└─ if (PARKING2_getNumero(hc) != proxAparcarNum[alg])
    └─ ERROR: Coche fuera de orden
    └─ ReleaseSemaphore(hOrden[alg], 1, NULL)
    └─ free(datos); return 1
└─
└─ proxAparcarNum[alg]++
    └─ Incrementar contador para el siguiente coche
└─
└─ PARKING2_aparcar(hc, &alg, aparcar_commit, permiso_avance, permiso_avance_commit)
    └─ Animar el aparcamiento del coche
        └─ Durante esta llamada se invocan las callbacks:
            - permiso_avance() (múltiples veces)
            - permiso_avance_commit()
            - aparcar_commit() (al final)
└─
└─ free(datos)
└─ return 0

```

Número de hilos: Variable (uno por coche que llegue y encuentre hueco) **Prioridad:** Heredada de la DLL **Ciclo de vida:** Desde que se crea hasta que PARKING2_aparcar() retorna

4.4. Hilos de Desaparcamiento (hilo_desaparcar)

Se crean dinámicamente cuando un coche expira su tiempo de aparcamiento.

```

hilo_desaparcar(DATOS_HILO *datos)
└─ HCoche hc ← datos->hc
└─ int alg ← datos->algoritmo
└─
└─ PARKING2_desaparcar(hc, &alg, permiso_avance, permiso_avance_commit)
    └─ Animar la salida del coche del aparcamiento
        └─ Durante esta llamada se invocan las callbacks:

```

```

|         - permiso_avance()      (múltiples veces)
|         - permiso_avance_commit()
|
|— free(datos)
|— return 0

```

Número de hilos: Variable (uno por coche que deba salir) **Prioridad:** Heredada de la DLL
Ciclo de vida: Desde que se crea hasta que PARKING2_desaparcacar() retorna **Orden:** No es necesario que salgan en orden numérico

5. Pseudocódigo de Operaciones Críticas

5.1. Secuencia de Aparcamiento

```

FUNCIÓN funcion_llegada_algoritmo(hc)
|— pos ← algoritmo_busqueda(hc)
|
|— SI pos >= 0 ENTONCES
|   |— datos ← malloc(DATOS_HILO)
|   |— datos->hc ← hc
|   |— datos->algoritmo ← algoritmo_id
|   |— hThread ← CreateThread(NULL, 0, hilo_aparcar, datos, 0, NULL)
|   |— SI hThread != NULL ENTONCES CloseHandle(hThread)
|   |— RETORNA pos
|
|— SINO (pos == -1, encolar)
|   |— RETORNA -1
FIN FUNCIÓN

FUNCIÓN hilo_aparcar(param)
|— datos ← (DATOS_HILO*)param
|— hc ← datos->hc
|— alg ← datos->algoritmo
|
|— WaitForSingleObject(hOrden[alg], INFINITE)
|   |— Bloquearse hasta que sea el turno
|
|— numCoche ← PARKING2_getNumero(hc)
|— SI numCoche != proxAparcarNum[alg] ENTONCES
|   |— ERROR: Coche fuera de orden
|   |— ReleaseSemaphore(hOrden[alg], 1, NULL)
|   |— free(datos)
|   |— RETORNA 1
|
|— proxAparcarNum[alg]++
|   |— Incrementar para el siguiente
|
|— PARKING2_aparcar(hc, &alg, aparcar_commit, ...)
|   |— Animar el aparcamiento (la DLL controla todo)
|
|— free(datos)
|— RETORNA 0
FIN FUNCIÓN

```

5.2. Callback: aparcar_commit

Se invoca cuando la DLL ha confirmado que el coche está completamente aparcado.

```

FUNCIÓN aparcar_commit(hc)
|— alg ← PARKING2_getAlgoritmo(hc)
|

```

```

└─ ReleaseSemaphore(hOrden[alg], 1, NULL)
  └─ Permitir que aparque el siguiente coche
└─ RETORNA
FIN FUNCIÓN

```

5.3. Callback: permiso_avance

Se invoca ANTES de que el coche avance. Debe bloquearse sin consumir CPU si no es seguro.

```

FUNCIÓN permiso_avance(hc)
└─ alg ← PARKING2_getAlgoritmo(hc)
└─ X1, Y1 ← posición actual
└─ X2, Y2 ← posición destino
└─ CASO 1: Avance horizontal en el carril (Y1 == Y2 == 2)
  └─ MIENTRAS VERDADERO HACER
    └─ WaitForSingleObject(hMutex[alg], INFINITE)
    └─ SI carril[alg][X2] == 0 ENTONCES
      └─ carril[alg][X2] ← 1          ← Ocupar posición
      └─ ReleaseMutex(hMutex[alg])
      └─ ROMPER
    └─ SINO
      └─ ReleaseMutex(hMutex[alg])
      └─ WaitForSingleObject(hAvance[alg], INFINITE)
      └─ CONTINUAR
    └─ FIN SI
└─ CASO 2: Avance acera a carretera (Y1 < Y2 == 2)
  └─ MIENTRAS VERDADERO HACER
    └─ WaitForSingleObject(hMutex[alg], INFINITE)
    └─ puede_ocupar ← VERDADERO
    └─ PARA i = X2 HASTA X2 + longitud - 1 HACER
      └─ SI carril[alg][i] != 0 ENTONCES
        └─ puede_ocupar ← FALSO
        └─ FIN SI
    └─ SI puede_ocupar ENTONCES
      └─ PARA i = X2 HASTA X2 + longitud - 1 HACER
        └─ carril[alg][i] ← 1
      └─ ReleaseMutex(hMutex[alg])
      └─ ROMPER
    └─ SINO
      └─ ReleaseMutex(hMutex[alg])
      └─ WaitForSingleObject(hAvance[alg], INFINITE)
      └─ CONTINUAR
    └─ FIN SI
└─ RETORNA
FIN FUNCIÓN

```

5.4. Callback: permiso_avance_commit

Se invoca DESPUÉS de que el coche ha avanzado. Libera recursos y despierta bloqueados.

```

FUNCIÓN permiso_avance_commit(hc)
├─ alg ← PARKING2_getAlgoritmo(hc)
├─ X_anterior, Y_anterior ← GetX2, GetY2 (posición de antes)
├─ Y_actual ← GetY (posición actual)
├─ longitud ← PARKING2_getLongitud(hc)
├─
├─ CASO 1: Desaparque acera (Y_anterior == 1, Y_actual == 2)
│   ├─ pos ← PARKING2_getPosiciónEnAcera(hc)
│   ├─ SI pos >= 0 ENTONCES
│   │   └─ PARA i = pos HASTA pos + longitud - 1 HACER
│   │       └─ acera[alg][i] ← 0 ← Liberar acera
│   └─
├─ CASO 2: Aparcamiento carretera (Y_anterior == 2, Y_actual == 1)
│   ├─ WaitForSingleObject(hMutex[alg], INFINITE)
│   ├─ PARA i = X_anterior HASTA X_anterior + longitud - 1 HACER
│   │   └─ carril[alg][i] ← 0 ← Liberar carril
│   ├─ ReleaseMutex(hMutex[alg])
│   └─ ReleaseSemaphore(hAvance[alg], 1, NULL) ← Despertar bloqueados
├─
├─ CASO 3: Avance horizontal carril (Y_anterior == 2, Y_actual == 2)
│   ├─ final_coche ← X_anterior + longitud - 1
│   ├─ SI final_coche EN RANGO ENTONCES
│   │   ├─ WaitForSingleObject(hMutex[alg], INFINITE)
│   │   ├─ carril[alg][final_coche] ← 0 ← Liberar posición anterior
│   │   ├─ ReleaseMutex(hMutex[alg])
│   │   └─ ReleaseSemaphore(hAvance[alg], 1, NULL)
│   └─
├─ RETORNA
FIN FUNCIÓN

```

6. Algoritmos de Búsqueda

6.1. Primer Ajuste

Criterio: Primer hueco disponible que quepa el coche.

```

FUNCIÓN primer_ajuste(hc)
├─ longitud ← PARKING2_getLongitud(hc)
├─ huecoLibre ← 0
├─ pos ← -1
├─
├─ PARA i = 0 HASTA TAM_PARKING - 1 HACER
│   ├─ SI acera[PRIMER_AJUSTE][i] == 0 ENTONCES
│   │   ├─ huecoLibre++
│   │   ├─ SI huecoLibre >= longitud ENTONCES
│   │   │   ├─ pos ← i - longitud + 1
│   │   │   └─ ROMPER
│   │   └─ SINO
│   │       └─ i++
│   └─ SINO
│       ├─ i += acera[PRIMER_AJUSTE][i] ← Optimización: saltar coche
│       └─ huecoLibre ← 0
├─
├─ SI pos >= 0 ENTONCES
│   └─ PARA j = pos HASTA pos + longitud - 1 HACER
│       └─ acera[PRIMER_AJUSTE][j] ← longitud
└─

```

```

└─ RETORNA pos
FIN FUNCIÓN

```

6.2. Siguiente Ajuste

Criterio: Búsqueda circular desde la última posición.

```

FUNCIÓN siguiente_ajuste(hc)
└─ longitud ← PARKING2_getLongitud(hc)
└─ huecoLibre ← 0
└─ pos ← -1
└─ inicio ← proxAparcar[SIGUIENTE_AJUSTE]
└─
└─ // Retroceder si la posición actual está libre
└─ SI acera[SIGUIENTE_AJUSTE][inicio] == 0 ENTONCES
    └─ MIENTRAS inicio > 0 Y acera[SIGUIENTE_AJUSTE][inicio - 1] == 0 HACER
        └─ inicio--
└─
└─ // Primera pasada: desde 'inicio' hasta el final
└─ PARA i = inicio HASTA TAM_PARKING - 1 HACER
    └─ SI acera[SIGUIENTE_AJUSTE][i] == 0 ENTONCES
        └─ huecoLibre++
        └─ SI huecoLibre >= longitud ENTONCES
            └─ pos ← i - longitud + 1
            └─ ROMPER
        └─ SINO
            └─ i += acera[SIGUIENTE_AJUSTE][i]
            └─ huecoLibre ← 0
└─
└─ // Segunda pasada: desde el inicio hasta 'inicio'
└─ SI pos == -1 ENTONCES
    └─ huecoLibre ← 0
    └─ PARA i = 0 HASTA inicio - 1 HACER
        └─ SI acera[SIGUIENTE_AJUSTE][i] == 0 ENTONCES
            └─ huecoLibre++
            └─ SI huecoLibre >= longitud ENTONCES
                └─ pos ← i - longitud + 1
                └─ ROMPER
            └─ SINO
                └─ i += acera[SIGUIENTE_AJUSTE][i]
                └─ huecoLibre ← 0
└─
└─ // Guardar datos
└─ SI pos >= 0 ENTONCES
    └─ PARA j = pos HASTA pos + longitud - 1 HACER
        └─ acera[SIGUIENTE_AJUSTE][j] ← longitud
    └─ proxAparcar[SIGUIENTE_AJUSTE] ← (pos + longitud) MOD TAM_PARKING
└─
└─ RETORNA pos
FIN FUNCIÓN

```

6.3. Mejor Ajuste

Criterio: Hueco más pequeño que quepa el coche.

```

FUNCIÓN mejor_ajuste(hc)
└─ longitud ← PARKING2_getLongitud(hc)
└─ pos ← -1
└─ huecoActual ← 0

```

```

└─ inicioHuecoActual ← -1
└─ mejorTamano ← TAM_PARKING + 1

└─ PARA i = 0 HASTA TAM_PARKING - 1 HACER
    └─ SI acera[MEJOR_AJUSTE][i] == 0 ENTONCES
        └─ SI huecoActual == 0 ENTONCES
            └─ inicioHuecoActual ← i
            └─ huecoActual++

        └─ SINO (hay un coche)
            └─ SI huecoActual >= longitud ENTONCES
                └─ SI huecoActual < mejorTamano ENTONCES
                    └─ mejorTamano ← huecoActual
                    └─ pos ← inicioHuecoActual
                └─ huecoActual ← 0

└─ // Evaluar el último hueco
└─ SI huecoActual >= longitud ENTONCES
    └─ SI huecoActual < mejorTamano ENTONCES
        └─ mejorTamano ← huecoActual
        └─ pos ← inicioHuecoActual

└─ // Guardar datos
└─ SI pos >= 0 ENTONCES
    └─ PARA j = pos HASTA pos + longitud - 1 HACER
        └─ acera[MEJOR_AJUSTE][j] ← longitud

└─ RETORNA pos
FIN FUNCIÓN

```

6.4. Peor Ajuste

Criterio: Hueco más grande disponible.

```

FUNCIÓN peor_ajuste(hc)
└─ longitud ← PARKING2_getLongitud(hc)
└─ pos ← -1
└─ huecoActual ← 0
└─ inicioHuecoActual ← -1
└─ peorTamano ← -1

└─ PARA i = 0 HASTA TAM_PARKING - 1 HACER
    └─ SI acera[PEOR_AJUSTE][i] == 0 ENTONCES
        └─ SI huecoActual == 0 ENTONCES
            └─ inicioHuecoActual ← i
            └─ huecoActual++

        └─ SINO (hay un coche)
            └─ SI huecoActual >= longitud ENTONCES
                └─ SI huecoActual > peorTamano ENTONCES
                    └─ peorTamano ← huecoActual
                    └─ pos ← inicioHuecoActual
                └─ huecoActual ← 0

└─ // Evaluar el último hueco
└─ SI huecoActual >= longitud ENTONCES
    └─ SI huecoActual > peorTamano ENTONCES
        └─ peorTamano ← huecoActual
        └─ pos ← inicioHuecoActual

```

```

└─ // Guardar datos
└─ SI pos >= 0 ENTONCES
    └─ PARA j = pos HASTA pos + longitud - 1 HACER
        └─ acera[PEOR_AJUSTE][j] ← longitud
└─ RETORNA pos
FIN FUNCIÓN

```

7. Tabla Resumen de Objetos de Sincronización

Objeto	Tipo	Inicial	Máximo	Uso
hMutex[i]	Mutex	Libre	1	Proteger acera[i] y carril[i]
hOrden[i]	Semáforo	1	1	Garantizar orden numérico (1,2,3...)
hAvance[i]	Semáforo	0	999	Sincronizar avances de coches

8. Consideraciones de Concurrencia

8.1. Puntos Críticos

1. **acera[alg][pos]**: Accedida por múltiples hilos sin mutex en lectura (rápida). Protegida en escritura por los propios algoritmos.
2. **carril[alg][pos]**: **Siempre** protegida por hMutex[alg] debido a contienda por posiciones.
3. **proxAparcarNum[alg]**: Protegida implícitamente por hOrden[alg]. Solo se incrementa dentro de hilo_aparcar() después de adquirir el semáforo.
4. **proxAparcar[alg]**: Solo se modifica dentro de siguiente_ajuste() cuando se encuentra un hueco.

8.2. Garantías de Sincronización

- **Orden de aparcamiento**: Garantizado por hOrden[alg] + proxAparcarNum[alg]
- **Sin colisiones en carril**: Garantizado por hMutex[alg] en permiso_avance() y permiso_avance_commit()
- **Sin consumo de CPU**: Los hilos se bloquean con WaitForSingleObject() sin spin-waiting
- **Prevención de deadlock**: No hay ciclos en adquisición de semáforos/mutex

9. Optimizaciones Realizadas

9.1. 1. Salto de Coches en Arrays

En los algoritmos de búsqueda, cuando se encuentra un coche ocupando posiciones:

```
// Versión ingenua
```

```
i++;
```

```
// Versión optimizada
```

```
i += acera[ALGORITMO][i]; // Salta directamente al final del coche
```

Mejora: Reduce iteraciones en $O(n^2)$ a $O(n)$.

9.2. 2. Retroceso en Siguiente Ajuste

```
if (acera[SIGUIENTE_AJUSTE][inicio] == 0) {  
    while (inicio > 0 && acera[SIGUIENTE_AJUSTE][inicio - 1] == 0) {  
        inicio--;  
    }  
}
```

Mejora: Encuentra el inicio real del hueco libre en lugar de comenzar en medio.

9.3. 3. Búsqueda Circular en Siguiente Ajuste

```
// Primera pasada: desde 'inicio' hasta fin  
// Segunda pasada: desde inicio hasta 'inicio'
```

Mejora: Reparte mejor los coches entre toda la acera.

9.4. 4. Cierre Inmediato de Manejadores de Hilos

```
HANDLE hThread = CreateThread(...);  
if (hThread != NULL) {  
    CloseHandle(hThread); // Liberar manejador inmediatamente  
}
```

Mejora: Evita agotamiento de manejadores en sistemas con muchos hilos.

10. Ejemplo de Ejecución

10.1. Escenario: Tres Coches Llegando

```
t=0.0s   Coche 1 llega (longitud=2)  
        └─ Hilo_llegada: aparcarse encontrado en pos=10  
            └─ CreateThread(hilo_aparcarse)  
                └─ WaitForSingleObject(hOrden[PA]) ✓ Adquirido  
                    └─ proxAparcarNum[PA] == 1 ✓  
                        └─ proxAparcarNum[PA]++ → 2  
                            └─ PARKING2_aparcarse()  
                                └─ permiso_avance() → Ocupar carril  
                                    └─ permiso_avance_commit() → Liberar  
                                        └─ aparcarse_commit()  
                                            └─ ReleaseSemaphore(hOrden[PA], 1)  
  
t=0.5s   Coche 2 llega (longitud=3)  
        └─ Hilo_llegada: aparcarse encontrado en pos=13  
            └─ CreateThread(hilo_aparcarse)  
                └─ WaitForSingleObject(hOrden[PA]) ✓ Adquirido (por Coche 1 liberado)  
                    └─ proxAparcarNum[PA] == 2 ✓  
                        └─ proxAparcarNum[PA]++ → 3  
                            └─ PARKING2_aparcarse()  
                                └─ ...  
  
t=1.0s   Coche 3 llega (longitud=1)  
        └─ Hilo_llegada: aparcarse encontrado en pos=17  
            └─ CreateThread(hilo_aparcarse)  
                └─ WaitForSingleObject(hOrden[PA]) (esperar)  
                    └─ Bloqueado esperando que termine Coche 2  
                        └─ Coche 2 termina: aparcarse_commit() → ReleaseSemaphore()  
                            └─ WaitForSingleObject(hOrden[PA]) ✓ Adquirido  
                                └─ proxAparcarNum[PA] == 3 ✓  
                                    └─ PARKING2_aparcarse()  
                                        └─ ...
```